

LAGMX: Automated preparation and simulation of single- and multi-ligand protein complexes in GROMACS

La Ode Aman¹✉, Arfan², Aiyi Asnawi³, Purnawan Pontana Putra⁴,
Netty Ino Ischak⁵, Dizky Ramadani Putri Papeo¹, Hamsidar Hasan¹,
and A. Mu'thi Andy Suryadi¹

¹ Department of Pharmacy, Faculty of Sports and Health, Universitas Negeri Gorontalo, Jl. Jenderal Sudirman No. 6, Kota Gorontalo, Gorontalo 96128, Indonesia ² Faculty of Pharmacy, Universitas Halu Oleo, Kendari, Southeast Sulawesi, Indonesia ³ Faculty of Pharmacy, Universitas Bhakti Kencana, Bandung, West Java, Indonesia ⁴ Department of Pharmaceutical Chemistry, Faculty of Pharmacy, Universitas Andalas, Padang 25163, Indonesia ⁵ Department of Chemistry, Faculty of Mathematics and Natural Sciences, Universitas Negeri Gorontalo, Gorontalo, Indonesia ✉ Corresponding author

DOI: 10.xxxxxx/draft

Software

- [Review](#) ✉
- [Repository](#) ✉
- [Archive](#) ✉

Editor: ✉

Submitted: 02 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

LAGMX is a single Python script that automates the preparation, equilibration, and production stages of a molecular dynamics (MD) simulation of a protein-ligand complex in GROMACS (Abraham et al., 2015). Preparing one such system by hand normally takes nine stages – receptor curation, ligand parameterization, topology merging, solvation, ion addition, energy minimization, index-group selection, and two stages of equilibration – several of them interactive. LAGMX runs all of them, plus a configurable production run, from one invocation, for every complex directory it is pointed at.

It is built specifically to handle **multiple ligands per complex correctly**: moleculetype name collisions between identical ligand copies, position-restraint includes attached at the wrong scope, duplicated [atomtypes] entries, and per-ligand net charge silently assumed to be neutral are handled automatically rather than discovered by the user through wrong runs. Ligand parameterization uses GAFF2 (Wang et al., 2004) with a configurable charge method: AM1-BCC via antechamber (Jakalian et al., 2002; Wang et al., 2006), or Gasteiger for fast iteration.

Statement of need

Protein-ligand MD is routine in structure-based drug discovery, but the setup cost per system is high and scales badly with the number of ligands. A researcher screening a compound series, or studying a receptor with several co-bound cofactors, repeats those stages for every complex. The multi-ligand failure modes are quiet rather than loud: GROMACS accepts a topology in which two identical ligand copies share a moleculetype name, and antechamber accepts a charged ligand as neutral. Both produce a simulation that runs to completion and is wrong. LAGMX is aimed at computational chemists and pharmacy researchers who need many such systems prepared reproducibly, and reduces the setup of an arbitrary number of complexes to editing one configuration file and issuing one command.

State of the field

Existing tools address parts of this workflow, but none combine config-driven end-to-end execution with first-class support for an arbitrary number of ligands per complex. CHARMM-GUI (Jo et al., 2008) generates ready-to-run input files for protein-ligand systems, including

multi-ligand ones, but does not run the pipeline: preparation happens on its servers, and equilibration and production are still launched and managed by the user. **acpype** (Sousa da Silva & Vranken, 2012) automates ligand topology generation, but does not merge topologies, solvate, equilibrate, or run production – one stage, not the pipeline. **BioExcel Building Blocks** (Andrio et al., 2019) provides composable Python wrappers around individual GROMACS and AmberTools commands; it gives users the blocks to construct a pipeline rather than a pipeline that runs by default, leaving assembly and validation – including the multi-ligand correctness issues above – to the user. LAGMX targets that gap.

Software design

LAGMX is deliberately a single ~900-line script rather than a package, depending only on GROMACS, AmberTools, ParmEd, and the Python standard library: the target user works on a shared HPC node or lab workstation where installing a package tree is friction rather than convenience. The cost is a tool harder to unit-test and to reuse programmatically – a prototype trade-off we expect to revisit. Configuration lives in one plain-text `gmx_config.txt` rather than in flags, so every parameter that affects the physics sits beside the results and stays recoverable months later. Stage resumption is derived from output files rather than a state database: `detect_stage()` tests for the final artefact of each stage (`em.gro`, `nvt.gro`, `npt.gro`, `md.cpt`), so an interrupted run repeats only that stage.

Two correctness decisions are worth naming: `build_ligand_ids()` gives each ligand file a unique moleculetype name, appending a numeric suffix on collision, and `detect_net_charge()` reads a PDB's formal-charge column or sums a MOL2 file's partial charges and passes the result to antechamber `-nc`. Both run before any GROMACS command executes, so a mis-specified system fails during preparation rather than yielding a trajectory that is wrong but plausible. This logic was solved separately and inconsistently across the six shell scripts the first author published in 2023, which LAGMX consolidates and supersedes.

Validation

LAGMX ships a version-controlled test matrix (`run_matrix/`) of seven scenarios: one receptor chain with one ligand; a receptor split across three files with three distinct ligands; three *identical* ligand copies (moleculetype deduplication); a ligand supplied as MOL2 instead of PDB; a charged ligand (ADP3-) alongside a neutral one; a deliberately malformed ligand, expected to fail cleanly rather than produce a silently wrong topology; and a five-ligand system mixing identical, distinct, and charged species. All six valid scenarios ran end to end – preparation through 5 ns of production – on a single consumer GPU (RTX 3070), with zero atom-name mismatches and zero LINCS constraint warnings. Per-scenario NPT temperature and density are in the project README.

Research impact statement

LAGMX has not yet been used outside the authors' group, and we claim no realized external impact; its case rests on near-term significance and on being ready for others to pick up. The validation matrix is the primary evidence: all seven scenarios, with their inputs and `.mdp` files, are committed, so any reader can reproduce the reported runs from a clean checkout. The software is MIT-licensed, tested in continuous integration across Python 3.10-3.12, and ships contribution guidelines. Multi-ligand protein-ligand MD in GROMACS is common enough in structure-based drug discovery that we expect the audience for a working end-to-end pipeline to be broader than our own group.

Limitations

LAGMX is at an early stage: a prototype, not yet used outside this project. Five nanoseconds of production demonstrates that the pipeline executes correctly, not that any given system's

dynamics have converged; grompp is still run with `-maxwarn` during equilibration and production; index-group selection assumes ligands are the only non-protein, non-solvent species present; and MOL2 ligand input is not guaranteed to be chemically complete – one test ligand required a missing hydrogen to be restored by a separate repair script before antechamber would accept it.

AI usage disclosure

Generative AI tools were used in preparing both the software and this manuscript. **Software.** AI coding assistants were used for parts of LAGMX.py – debugging, drafting individual functions, and clarifying GROMACS and AmberTools command usage; the architecture and the majority of the implementation were written by the authors. **Manuscript.** AI assistance was used for parts of the prose here and in the project README; in the revision that produced this version, an AI coding assistant (Claude, Anthropic) drafted the State of the field, Software design, Research impact statement, and AI usage disclosure sections, rewrote the Statement of need, and authored the continuous integration workflow that compiles the manuscript. **Verification.** All AI-assisted output was reviewed by the authors: design decisions checked against the source code, validation results against the simulation outputs, and each reference against its DOI. The authors take full responsibility for the content of this paper and of the software.

Acknowledgements

This research received no external funding; it was self-funded by the authors.

References

- Abraham, M. J., Murtola, T., Schulz, R., Páll, S., Smith, J. C., Hess, B., & Lindahl, E. (2015). GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers. *SoftwareX*, 1-2, 19–25. <https://doi.org/10.1016/j.softx.2015.06.001>
- Andrio, P., Hospital, A., Conejero, J., Jordá, L., Del Pino, M., Codo, L., Soiland-Reyes, S., Goble, C., Lezzi, D., Badia, R. M., Orozco, M., & Gelpí, J. Ll. (2019). BioExcel building blocks, a software library for interoperable biomolecular simulation workflows. *Scientific Data*, 6, 169. <https://doi.org/10.1038/s41597-019-0177-4>
- Jakalian, A., Jack, D. B., & Bayly, C. I. (2002). Fast, efficient generation of high-quality atomic charges. AM1-BCC model: II. Parameterization and validation. *Journal of Computational Chemistry*, 23(16), 1623–1641. <https://doi.org/10.1002/jcc.10128>
- Jo, S., Kim, T., Iyer, V. G., & Im, W. (2008). CHARMM-GUI: A web-based graphical user interface for CHARMM. *Journal of Computational Chemistry*, 29(11), 1859–1865. <https://doi.org/10.1002/jcc.20945>
- Sousa da Silva, A. W., & Vranken, W. F. (2012). ACPYPE - AnteChamber PYthon parser interface. *BMC Research Notes*, 5, 367. <https://doi.org/10.1186/1756-0500-5-367>
- Wang, J., Wang, W., Kollman, P. A., & Case, D. A. (2006). Automatic atom type and bond type perception in molecular mechanical calculations. *Journal of Molecular Graphics and Modelling*, 25(2), 247–260. <https://doi.org/10.1016/j.jmgm.2005.12.005>
- Wang, J., Wolf, R. M., Caldwell, J. W., Kollman, P. A., & Case, D. A. (2004). Development and testing of a general amber force field. *Journal of Computational Chemistry*, 25(9), 1157–1174. <https://doi.org/10.1002/jcc.20035>